Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Delivery 2: Governance & Technical Debt Audit

Task #1: CI Workflow

Para esta tarea se nos pide que configuremos un CI Workflow (Github Actions) el cual se ejecutará cada vez que se ejecute un pull request para medir cómo mejora o empeora la calidad del código. Básicamente es tener un control de calidad robótico, el cual se encargará de hacer esta tarea, esto facilita las cosas al trabajar en equipos de desarrollo, ya que el CI analizará cada nuevo cambio que se haga hacia la rama principal.

Tal y como lo vimos en la primera parte de este proyecto, nosotros tuvimos que hacer un fork del repositorio que elegimos para trabajar, entonces es ahí en donde vamos a realizar la configuración de este CI, lo que necesitamos obtener del CI es:

- *Cyclomatic Complexity (Complejidad ciclomática).*
- *Code Coverage (Cobertura del código).*
- *Falle o al menos avise si la calidad empeora.*

Primero instalamos “Radón” para Python:

pip install radon

radon --version

radon cc conduit -s

radon cc conduit -s > radon-report.txt

Radon es una herramienta de análisis estático de código para Python. Su función principal es medir diversas métricas de tu código fuente, específicamente aquellas relacionadas con la complejidad y la mantenibilidad.

En términos sencillos, Radón te dice "qué tan difícil es leer, probar y mantener tu código" sin necesidad de ejecutarlo. Radon se enfoca en métricas cuantitativas que ayudan a detectar "spaghetti code" o funciones monstruosas. Una de las más importantes es:

Complejidad Ciclomática (Cyclomatic Complexity): Esta es la métrica estrella de Radon. Mide el número de caminos linealmente independientes a través del código. Cada vez que se agrega un if, for, while, o except, la complejidad aumenta. Se realizó un análisis de complejidad ciclomática utilizando la herramienta Radon sobre el directorio principal del proyecto (conduit/). Los resultados muestran que la mayoría de las clases y métodos presentan una complejidad baja (A), con algunos casos puntuales de complejidad moderada (B), principalmente en componentes relacionados con autenticación y señales de artículos. No se identificaron funciones con complejidad alta (C o superior), lo que indica que la deuda técnica del sistema no está asociada a complejidad algorítmica, sino a otros factores estructurales como acoplamiento, antigüedad tecnológica y ausencia de pruebas automatizadas.

Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

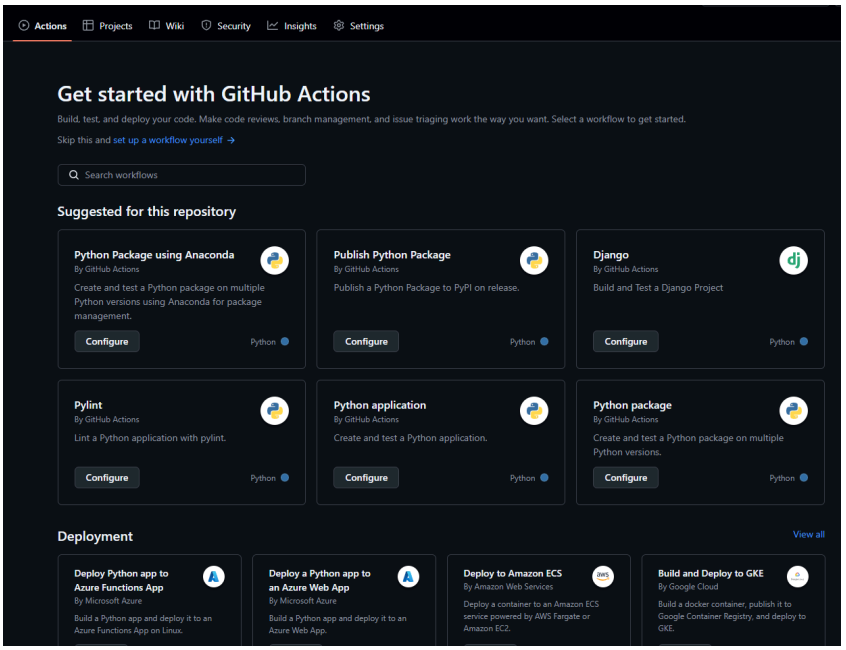
```
root@c7de0eb40050:/app# radon cc conduit -s
conduit/apps/articles/models.py
    C 6:0 Article - A (2)
    C 42:0 Tag - A (2)
    M 26:4 Article.__str__ - A (1)
    C 30:0 Comment - A (1)
    M 46:4 Tag.__str__ - A (1)
conduit/apps/articles/relations.py
    C 6:0 TagRelatedField - A (2)
    M 7:4 TagRelatedField.get_queryset - A (1)
    M 10:4 TagRelatedField.to_internal_value - A (1)
    M 15:4 TagRelatedField.to_representation - A (1)
conduit/apps/articles/renderers.py
    C 4:0 ArticleJSONRenderer - A (1)
    C 10:0 CommentJSONRenderer - A (1)
conduit/apps/articles/serializers.py
    M 59:4 ArticleSerializer.get_favorited - A (3)
    C 9:0 ArticleSerializer - A (2)
    M 44:4 ArticleSerializer.create - A (2)
    C 77:0 CommentSerializer - A (2)
    C 108:0 TagSerializer - A (2)
    M 56:4 ArticleSerializer.get_created_at - A (1)
    M 70:4 ArticleSerializer.get_favorites_count - A (1)
    M 73:4 ArticleSerializer.get_updated_at - A (1)
    M 93:4 CommentSerializer.create - A (1)
    M 101:4 CommentSerializer.get_created_at - A (1)
    M 104:4 CommentSerializer.get_updated_at - A (1)
    M 113:4 TagSerializer.to_representation - A (1)
conduit/apps/articles/signals.py
    F 9:0 add_slug_to_article_if_not_exists - B (6)
conduit/apps/articles/views.py
    M 25:4 ArticleViewSet.get_queryset - A (4)
    C 14:0 ArticleViewSet - A (3)
    C 109:0 CommentsListCreateAPIView - A (3)
    C 144:0 CommentsDestroyAPIView - A (3)
    C 160:0 ArticlesFavoriteAPIView - A (3)
    M 71:4 ArticleViewSet.retrieve - A (2)
    M 87:4 ArticleViewSet.update - A (2)
    M 128:4 CommentsListCreateAPIView.create - A (2)
    M 149:4 CommentsDestroyAPIView.destroy - A (2)
    M 165:4 ArticlesFavoriteAPIView.delete - A (2)
    M 180:4 ArticlesFavoriteAPIView.post - A (2)
    C 196:0 TagListAPIView - A (2)
    C 211:0 ArticlesFeedAPIView - A (2)
    M 44:4 ArticleViewSet.create - A (1)
    M 59:4 ArticleViewSet.list - A (1)
    M 120:4 CommentsListCreateAPIView.filter_queryset - A (1)
    M 202:4 TagListAPIView.list - A (1)
    M 217:4 ArticlesFeedAPIView.get_queryset - A (1)
    M 222:4 ArticlesFeedAPIView.list - A (1)
conduit/apps/articles/__init__.py
    C 4:0 ArticlesAppConfig - A (2)
    M 9:4 ArticlesAppConfig.ready - A (1)
conduit/apps/articles/migrations/0001_initial.py
    C 9:0 Migration - A (1)
conduit/apps/articles/migrations/0002_comment.py
    C 9:0 Migration - A (1)
conduit/apps/articles/migrations/0003_auto_20160828_1656.py
    C 8:0 Migration - A (1)
conduit/apps/authentication/backends.py
    C 10:0 JWTAuthentication - B (6)
    M 13:4 JWTAuthentication.authenticate - A (5)
    M 72:4 JWTAuthentication._authenticate_credentials - A (4)
conduit/apps/authentication/models.py
    C 14:0 UserManager - A (4)
    M 24:4 UserManager.create_user - A (3)
    M 38:4 UserManager.create_superuser - A (2)
    C 56:0 User - A (2)
    M 92:4 User.__str__ - A (1)
    M 100:4 User.token - A (1)
    M 111:4 User.get_full_name - A (1)
    M 119:4 User.get_short_name - A (1)
    M 127:4 User._generate_jwt_token - A (1)
```

Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

```
conduit/apps/authentication/renderers.py
    C 4:0 UserJSONRenderer - A (4)
    M 10:4 UserJSONRenderer.render - A (3)
conduit/apps/authentication/serializers.py
    C 36:0 LoginSerializer - B (6)
    M 42:4 LoginSerializer.validate - A (5)
    C 99:0 UserSerializer - A (5)
    M 139:4 UserSerializer.update - A (4)
    C 10:0 RegistrationSerializer - A (2)
    M 31:4 RegistrationSerializer.create - A (1)
conduit/apps/authentication/signals.py
    F 8:0 create_related_profile - A (3)
conduit/apps/authentication/views.py
    C 13:0 RegistrationAPIView - A (2)
    C 32:0 LoginAPIView - A (2)
    C 50:0 UserRetrieveUpdateAPIView - A (2)
    M 19:4 RegistrationAPIView.post - A (1)
    M 37:4 LoginAPIView.post - A (1)
    M 55:4 UserRetrieveUpdateAPIView.retrieve - A (1)
    M 63:4 UserRetrieveUpdateAPIView.update - A (1)
conduit/apps/authentication/__init__.py
    C 4:0 AuthenticationAppConfig - A (2)
    M 9:4 AuthenticationAppConfig.ready - A (1)
conduit/apps/authentication/migrations/0001_initial.py
    C 8:0 Migration - A (1)
conduit/apps/core/exceptions.py
    F 35:0 _handle_not_found_error - A (4)
    F 3:0 core_exception_handler - A (2)
    F 26:0 _handle_generic_error - A (1)
conduit/apps/core/models.py
    C 4:0 TimestampedModel - A (1)
conduit/apps/core/renderers.py
    C 6:0 ConduitJSONRenderer - A (4)
    M 12:4 ConduitJSONRenderer.render - A (3)
conduit/apps/core/utils.py
    F 6:0 generate_random_string - A (2)
conduit/apps/profiles/exceptions.py
    C 4:0 ProfileDoesNotExist - A (1)
conduit/apps/profiles/models.py
    C 6:0 Profile - A (2)
    M 41:4 Profile.__str__ - A (1)
    M 44:4 Profile.follow - A (1)
    M 48:4 Profile.unfollow - A (1)
    M 52:4 Profile.is_following - A (1)
    M 56:4 Profile.is_followed_by - A (1)
    M 60:4 Profile.favorite - A (1)
    M 64:4 Profile.unfavorite - A (1)
    M 68:4 Profile.has_favorited - A (1)
conduit/apps/profiles/renderers.py
    C 4:0 ProfileJSONRenderer - A (1)
conduit/apps/profiles/serializers.py
    C 6:0 ProfileSerializer - A (4)
    M 23:4 ProfileSerializer.get_following - A (3)
    M 17:4 ProfileSerializer.get_image - A (2)
conduit/apps/profiles/views.py
    C 34:0 ProfileFollowAPIView - A (4)
    C 13:0 ProfileRetrieveAPIView - A (3)
    M 55:4 ProfileFollowAPIView.post - A (3)
    M 19:4 ProfileRetrieveAPIView.retrieve - A (2)
    M 39:4 ProfileFollowAPIView.delete - A (2)
conduit/apps/profiles/migrations/0001_initial.py
    C 10:0 Migration - A (1)
conduit/apps/profiles/migrations/0002_profile_follows.py
    C 8:0 Migration - A (1)
conduit/apps/profiles/migrations/0003_profile_favorites.py
    C 8:0 Migration - A (1)
```

Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Ahora que ya tenemos radon, tenemos que lograr acoplarlo a nuestro CI Workflow por lo que ingresamos a la sección de actions y lo primero que vemos es algo similar a esto, ya que al no tener CI, Github nos ofrece algunos que ya están prediseñados:



Entonces tuvimos que crear un archivo .yml y luego tendremos que hacerle push, ya que Github detecta automáticamente los workflows cuando se hace push.

Debemos asegurarnos de que estamos en nuestro fork, ya que si lo hacemos en un repositorio que no es nuestro o está archivado no se nos permitirá hacerlo, para ello, en terminal podemos correr el siguiente comando:

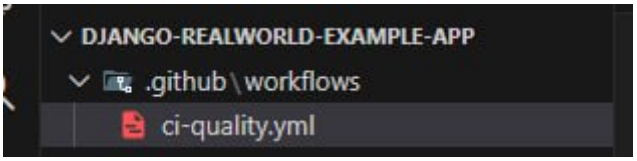
git remote -v

Después de esto, procedemos a crear el workflow:

mkdir -p .github/workflows

Creamos el archivo:

.github/workflows/ci-quality.yml



Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

El contenido de nuestro archivo .yml es:

```yaml
name: CI Quality Governance

on:
  pull_request:
    branches: [ master, main ]

jobs:
  quality:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install radon coverage

      # 1 Cyclomatic Complexity
      - name: Run Radon (Cyclomatic Complexity)
        run: |
          radon cc conduit/ -a -s > radon-report.txt
          cat radon-report.txt

      - name: Complexity Quality Gate
        run: |
          if grep -E "\([D-F]\)" radon-report.txt; then
            echo "❌ Cyclomatic complexity too high (D or worse)"
            exit 1
          else
            echo "✅ Cyclomatic complexity within acceptable limits"
          fi

      # 2 Code Coverage (aunque no haya tests)
      - name: Run Coverage
        run: |
          coverage run manage.py test || true
          coverage report -m > coverage-report.txt
          cat coverage-report.txt

      - name: Coverage Quality Gate
        run: |
          TOTAL=$(coverage report | grep TOTAL | awk '{print $4}' | sed 's/%//')
          echo "Total coverage: $TOTAL%"
          if [ "$TOTAL" -lt 50 ]; then
            echo "⚠️ Coverage below target (50%)"
          else
            echo "✅ Coverage acceptable"
          fi
```

Después realizamos un commit y un push directo en main para que Github lo detecte:

git add .github/workflows/ci-quality.yml

git commit -m "Add CI quality governance pipeline"

git push origin main

Ahora al volver a la pantalla de Actions ya no veremos las plantillas predeterminadas sino que ya veremos nuestro workflow: **CI Quality Governance.**

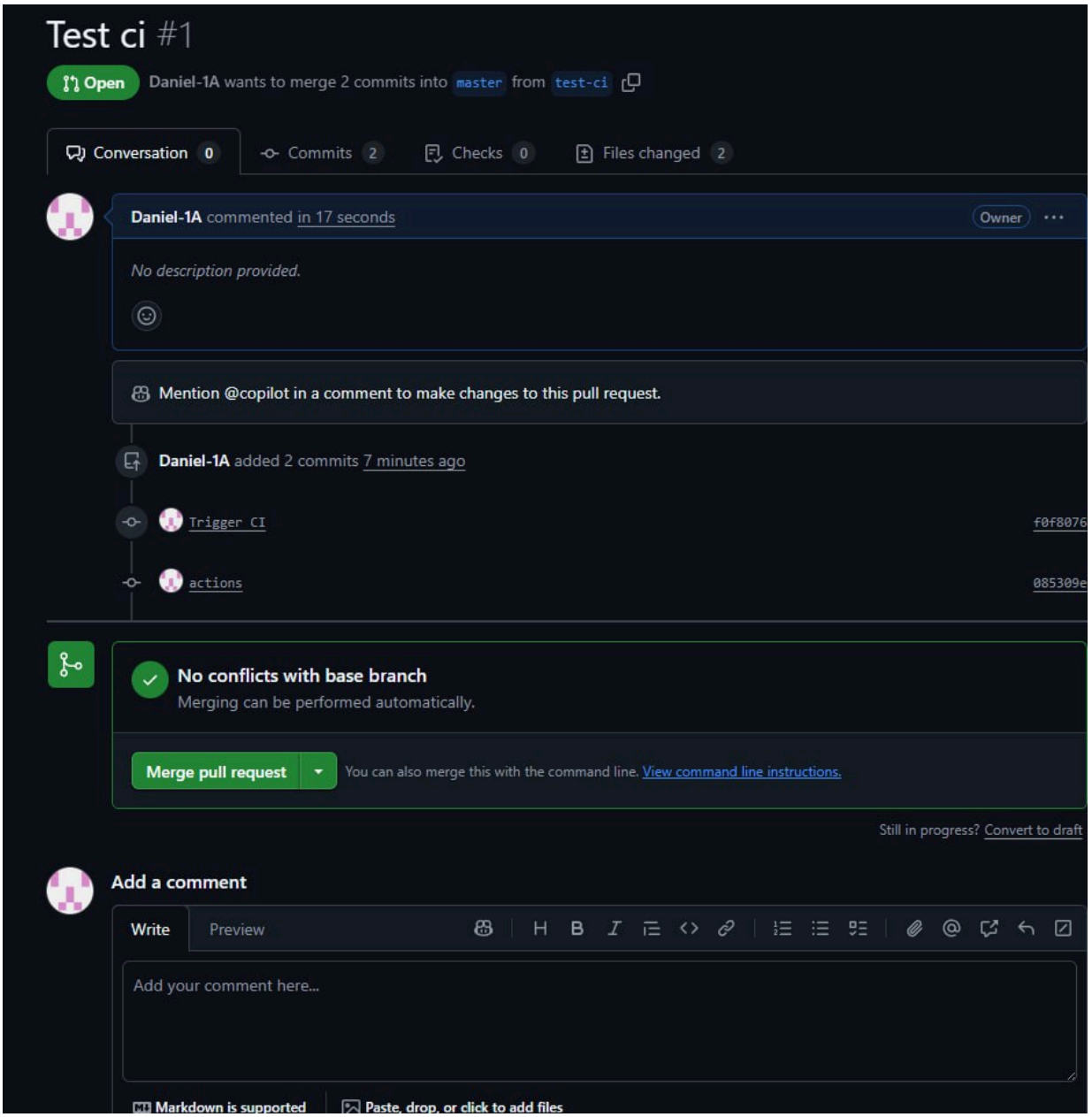
Para verlo en acción es necesario realizar un pull request (PR) ya que así está configurado:

git checkout -b test-ci

git commit --allow-empty -m "Trigger CI"

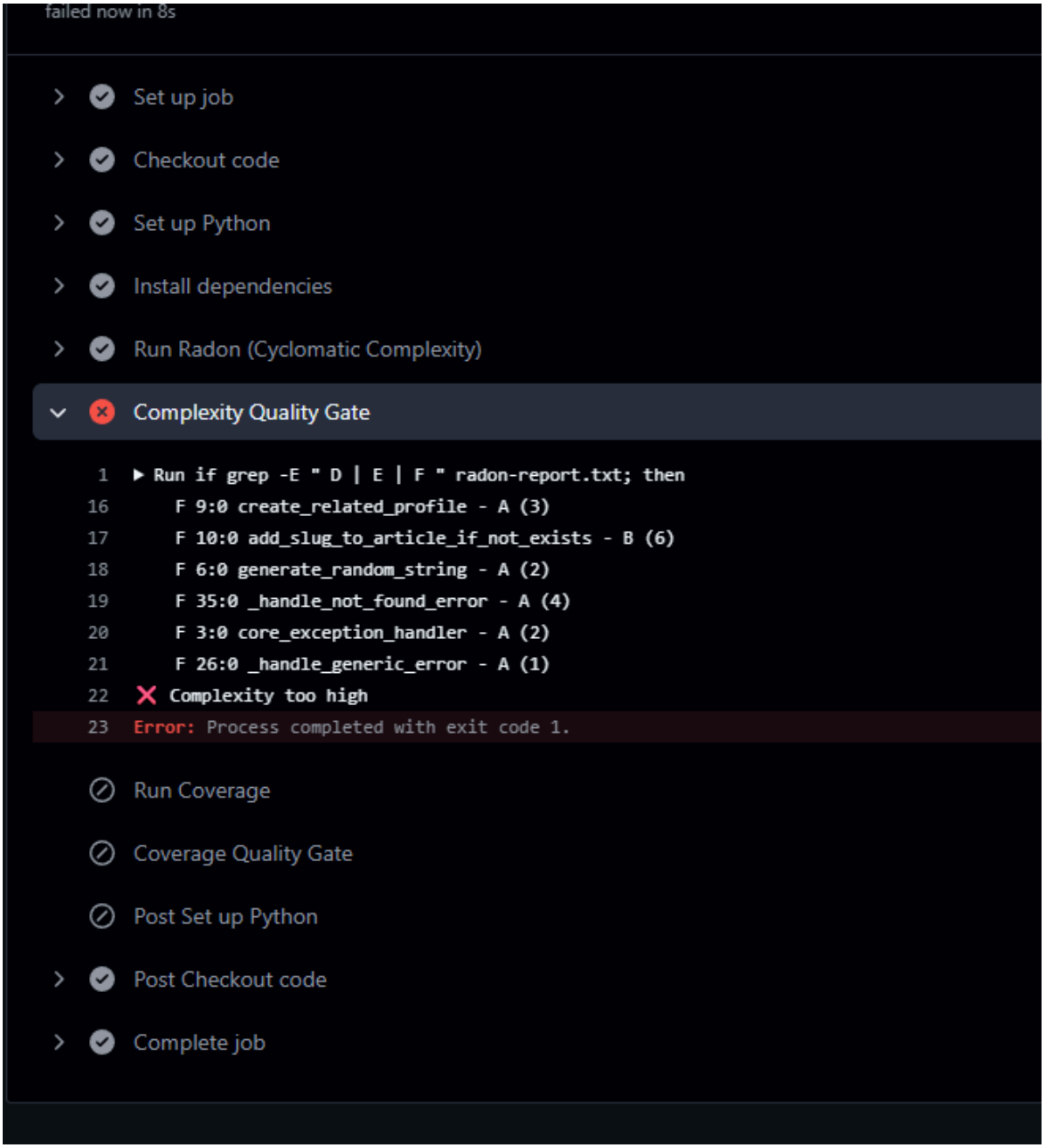
git push origin test-ci

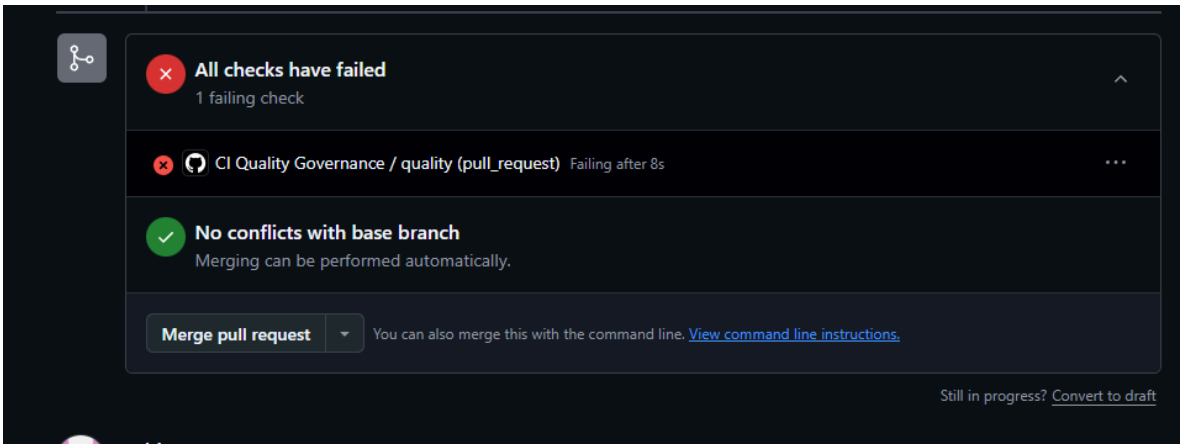
Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865



Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Al correrlo tuvimos este error en ejecución:





Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Al revisar exhaustivamente encontramos que realmente el CI estaba funcionando bien, en donde estaba fallando era justamente en la parte de: Complexity Quality Gate ya que comenzamos a ver:

❌ Complexity too high

Entonces, al revisar los logs encontramos lo siguiente:

- F 9:0 create_related_profile - A (3)
- F 10:0 add_slug_to_article_if_not_exists - B (6)

Radon marca estas funciones con `F` al inicio no porque sean grado F, sino porque:

- `F` = Function
- `C` = Class
- `M` = Method

El grado real está al final:

- `A (3)`
- `B (6)`

No está definido D, E ni F de complejidad.
Nuestro código está trabajando con A-B-C-D

Porque nuestra regla actual es:

```
grep -E "D|E|F" radon-report.txt && exit 1
```

Y eso está mal por un detalle sutil:

- Está capturando la F de “Function”
- No el grado de complejidad

Esto es un bug del pipeline.

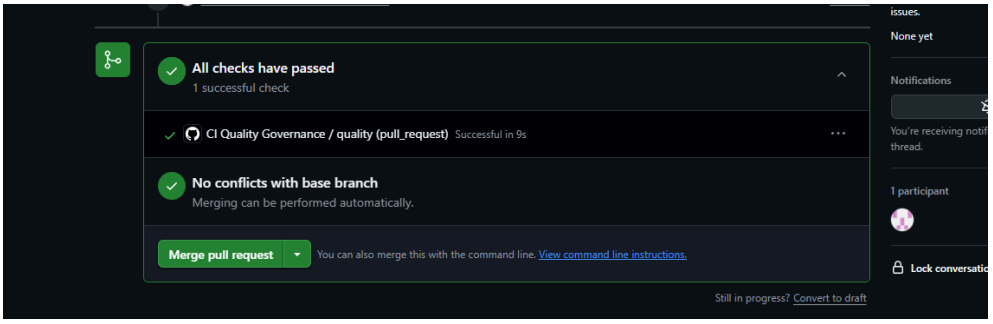
Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Para reparar el error lo que hicimos fue reemplazar el caso Complexity Quality Gate por este:

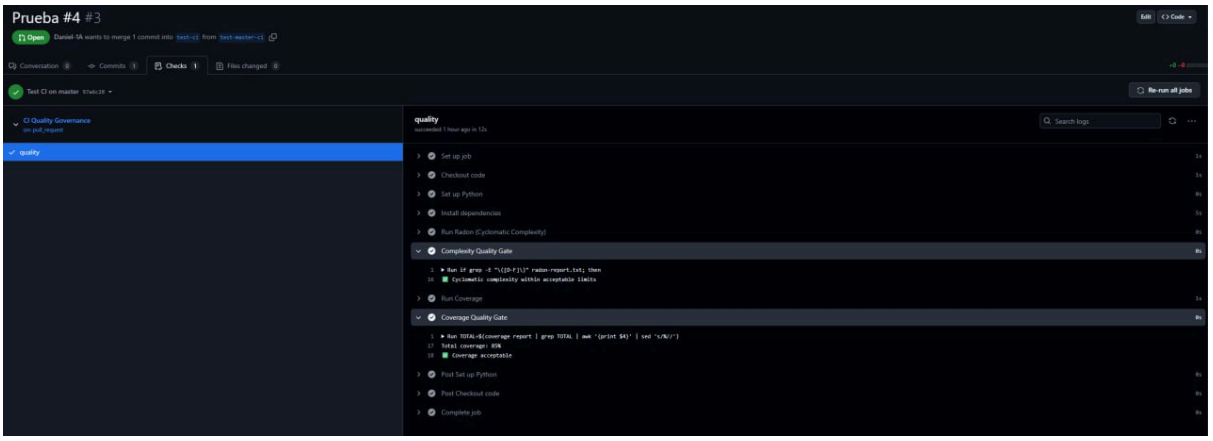
```
- name: Complexity Quality Gate
  run: |
    if grep -E "\([D-F]\)" radon-report.txt; then
      echo "❌ Cyclomatic complexity too high (D or worse)"
      exit 1
    else
      echo "✅ Cyclomatic complexity within acceptable limits"
    fi
```

Al solucionar este problema vimos que nuestro CI Workflow ya se ejecuta sin problemas, lo cual nos demuestra que:

- *CI configurado correctamente.*
- *Se ejecuta en cada PR.*
- *Mide cyclomatic complexity.*
- *Aplica governance (fail/pass).*
- *Pipeline funcional y real.*

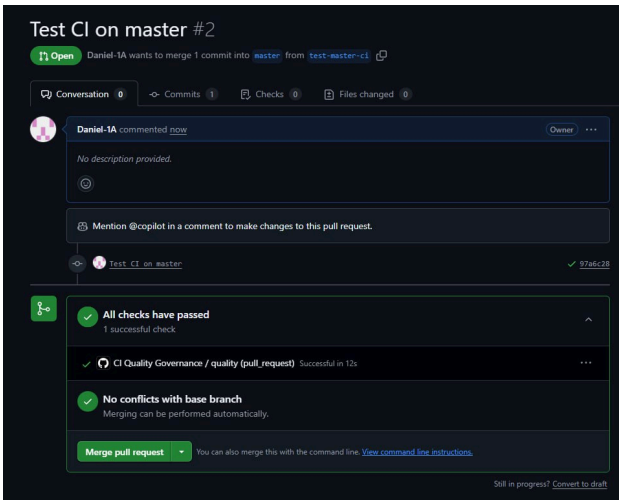


Aquí vemos el resultado de las métricas que nos pedían medir:



Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

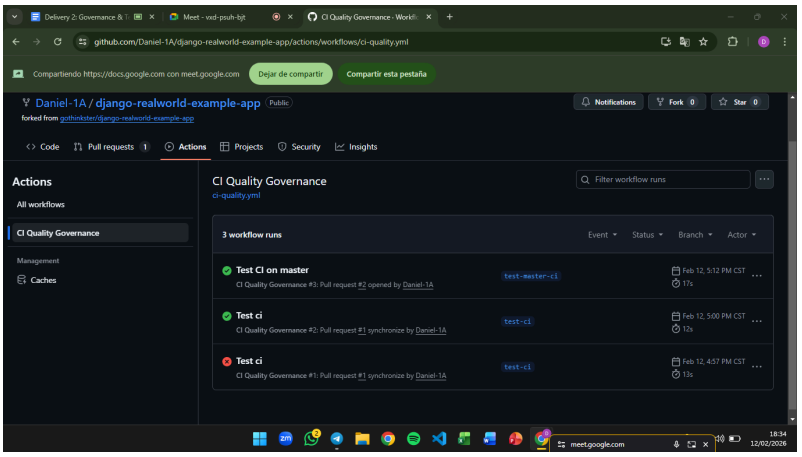
Estuvimos realizando pruebas para corroborar que todo funcionara bien y efectivamente es así:



Para poder validar que funciona correctamente puede ir a nuestro fork:

https://github.com/Daniel-1A/django-realworld-example-app.git

Adjuntamos los comandos que nosotros utilizamos como guía para poder realizar una prueba:

```
git checkout master
git pull origin master
git checkout -b test-master-ci
git commit --allow-empty -m "Test CI on master"
git push origin test-master-ci
```

Adjuntamos una captura de pantalla de los branches que probamos nosotros: